

IO_CAN.h File Reference

IO Driver functions for CAN communication. [More...](#)

```
#include "IO_Driver.h"
```

Data Structures

struct [IO_CAN_DATA_FRAME](#)

CAN data frame. [More...](#)

Macros

Message buffer direction

Selects the transmission direction of a CAN message buffer

```
#define IO\_CAN\_MSG\_READ 0U
```

```
#define IO\_CAN\_MSG\_WRITE 1U
```

CAN frame format

Selects the format for a CAN frame

```
#define IO\_CAN\_STD\_FRAME 0U
```

```
#define IO\_CAN\_EXT\_FRAME 1U
```

CAN baudrate

Selects the baudrate for a CAN channel

```
#define IO\_CAN\_BIT\_USER 0U
```

```
#define IO\_CAN\_BIT\_50\_KB 50U
```

```
#define IO\_CAN\_BIT\_100\_KB 100U
```

```
#define IO\_CAN\_BIT\_125\_KB 125U
```

```
#define IO\_CAN\_BIT\_250\_KB 250U
```

```
#define IO\_CAN\_BIT\_500\_KB 500U
```

```
#define IO\_CAN\_BIT\_1000\_KB 1000U
```

Functions

IO_ErrorType [IO_CAN_Init](#) ([ubyte1](#) channel, [ubyte2](#) baudrate, [ubyte1](#) tseg1, [ubyte1](#) tseg2, [ubyte1](#) sjw, [ubyte1](#) brp)

Initialization of the CAN communication driver. [More...](#)

IO_ErrorType [IO_CAN_DeInitHandle](#) ([ubyte2](#) handle)

Deinitializes a single message handle. [More...](#)

IO_ErrorType	IO_CAN_DeInit (ubyte1 channel)
	Deinitializes the given CAN channel. More...
IO_ErrorType	IO_CAN_ConfigMsg (ubyte2 *const handle, ubyte1 channel, ubyte1 mode, ubyte1 id_format, ubyte4 id, ubyte4 ac_mask)
	Configures a message object for the given CAN channel. More...
IO_ErrorType	IO_CAN_ReadMsg (ubyte2 handle, IO_CAN_DATA_FRAME *const buffer)
	Reads a message from a given message object. More...
IO_ErrorType	IO_CAN_WriteMsg (ubyte2 handle, const IO_CAN_DATA_FRAME *const data)
	Transmits a CAN message, using the given channel and message object. More...
IO_ErrorType	IO_CAN_Status (ubyte1 channel, ubyte1 *const rx_error_counter, ubyte1 *const tx_error_counter)
	Returns the error counters of the CAN channel. More...
IO_ErrorType	IO_CAN_MsgStatus (ubyte2 handle)
	Returns the status of a message buffer object. More...
IO_ErrorType	IO_CAN_ConfigFIFO (ubyte2 *const handle, ubyte1 channel, ubyte1 size, ubyte1 id_format, ubyte4 id, ubyte4 ac_mask)
	Configures a receive FIFO buffer for the given CAN channel. More...
IO_ErrorType	IO_CAN_ReadFIFO (ubyte2 handle, IO_CAN_DATA_FRAME *const buffer, ubyte1 buffer_size, ubyte1 *const rx_frames)
	Reads the data from a FIFO buffer. More...
IO_ErrorType	IO_CAN_FIFOStatus (ubyte2 handle)
	Returns the status of a FIFO buffer. More...

Detailed Description

IO Driver functions for CAN communication.

The CAN driver uses the DCAN module of the TMS570 CPU.

The CAN driver supports up to 7 CAN channels. The number of available channels depends on the used HW variant.

All CAN nodes have an own set of message objects and can not be allocated to other CAN nodes. For further details on message objects see [CAN handles and message objects](#).

CAN-API Usage:

- [Examples for CAN API functions](#)

Usage of the acceptance masks

The acceptance mask defines the relevant bits of the CAN ID. A binary 1 marks a relevant bit in the CAN ID on the same position.

Setting all bits of the acceptance mask (0x1FFFFFFF) only accepts the ID set with the ID parameter and rejects all other IDs.

Setting the acceptance mask to 0 causes the message buffer to accept any IDs.

Using this mechanism a message buffer can be used to accept a range of CAN IDs.

Example for an 11-bit ID:

```
ac_mask = 0x00000700 = 0 b 0 0000 0000 0000 0000 0111 0000 0000
id      = 0x00000200 = 0 b 0 0000 0000 0000 0000 0010 0000 0000
```

in this example all messages with an ID between 0x200 and 0x2FF are accepted.

CAN handles and message objects

The DCAN module of the TMS570 CPU provides a certain number of so called *CAN message objects*. A message object is a dedicated memory area which is used to store a CAN message. Every message object has its own identifier and acceptance mask. The number of available message objects depends on the CAN channel:

CAN channel	message objects
IO_CAN_CHANNEL_0	64 for receive and transmit
IO_CAN_CHANNEL_1	64 for receive and transmit
IO_CAN_CHANNEL_2	64 for receive and transmit
IO_CAN_CHANNEL_3	24 for receive, 8 for transmit
IO_CAN_CHANNEL_4	24 for receive, 8 for transmit
IO_CAN_CHANNEL_5	24 for receive, 8 for transmit
IO_CAN_CHANNEL_6	24 for receive, 8 for transmit

For receiving and transmitting CAN messages a message object needs to be configured. During the configuration the driver will return a so called *CAN handle*.

The *CAN handle* is a reference to the configured message object and is used to exchange data with the driver. There are two types of CAN handles:

- Standard message handles for receive and transmit
- FIFO buffer handles for receive and transmit

The number of CAN handles is identical to the number of message objects.

Standard message handles

A standard message handle is configured by calling the driver function `IO_CAN_ConfigMsg()`. It references to a single message object which can store *one* CAN message.

For receiving/transmitting data via a single message object the driver functions `IO_CAN_ReadMsg()` and `IO_CAN_WriteMsg()` are provided.

The status of a standard message handle can be checked with the function `IO_CAN_MsgStatus()`.

FIFO buffer handles

Attention

The driver supports FIFO buffers only for receive direction.

A FIFO buffer can be created by linking several message objects to one FIFO buffer which is able to store multiple CAN message.

A FIFO buffer handle is configured by calling the function `IO_CAN_ConfigFIFO()`. The desired size of the buffer is passed to the function via a parameter.

For receiving data via FIFO buffers the driver provides the function `IO_CAN_ReadFIFO()`. With this function multiple CAN messages can be retrieved with one function call.

The status of a FIFO buffer handle can be checked with the function `IO_CAN_FIFOStatus()`.

CAN Code Examples

Example for using the CAN API

Examples for CAN initialization

```
ubyte2 handle_w, handle_r;

IO_CAN_Init(IO_CAN_CHANNEL_0,
            IO_CAN_BIT_500_KB,
            0,          // default
            0,          // default
            0,          // default
            0);         // default

// standard message buffers

IO_CAN_ConfigMsg(&handle_w,
                 IO_CAN_CHANNEL_0, // channel 0
                 IO_CAN_MSG_WRITE, // transmit message buffer
                 IO_CAN_STD_FRAME, // standard ID
                 0,
                 0);

IO_CAN_ConfigMsg(&handle_r,
                 IO_CAN_CHANNEL_0, // channel 0
                 IO_CAN_MSG_READ,  // receive message buffer
                 IO_CAN_STD_FRAME, // standard ID
                 1,
                 0x1FFFFFFF);      // accept only ID 1
```

Example for CAN task function call

```
IO_CAN_DATA_FRAME can_frame;

// check if new message has been received
if (IO_CAN_MsgStatus(handle_r) == IO_E_OK)
{
    // if message has been received, read the message from the buffer
    IO_CAN_ReadMsg(handle_r,
                   &can_frame);

    // received message is now stored in can_frame
    // and can be used by the application
}

// assemble CAN frame:
can_frame.id = 1;
can_frame.id_format = IO_CAN_STD_FRAME;
can_frame.length = 4;
can_frame.data[0] = 1;
can_frame.data[1] = 2;
can_frame.data[2] = 3;
can_frame.data[3] = 4;
```

```
// transmit message
IO_CAN_WriteMsg(handle_w,
                &can_frame);

// wait until the transmission has been finished:
while (IO_CAN_MsgStatus(handle_w) != IO_E_OK);
```

Example for CAN FIFO usage

```
ubyte2 handle_fifo;
ubyte1 rx_count;
IO_CAN_DATA_FRAME fifo_frame[20];

// FIFO message buffer initialization

IO_CAN_ConfigFIFO(&handle_fifo,
                 IO_CAN_CHANNEL_0, // channel 0
                 20,               // 20 items
                 IO_CAN_STD_FRAME, // standard ID
                 0x700,            // accept all IDs from
                 0x700);           // 0x700 - 0x7FF

while (1)
{
    // Read FIFO buffer
    IO_CAN_ReadFIFO(handle_fifo,
                   &fifo_frame[0],
                   sizeof(fifo_frame) / sizeof(IO_CAN_DATA_FRAME),
                   &rx_count);

    if (rx_count > 0)
    {
        // process received frames
    }
}
```

Macro Definition Documentation

#define IO_CAN_BIT_1000_KB 1000U

Configure CAN Channel with 1000 kBit/s

#define IO_CAN_BIT_100_KB 100U

Configure CAN Channel with 100 kBit/s

#define IO_CAN_BIT_125_KB 125U

Configure CAN Channel with 125 kBit/s

#define IO_CAN_BIT_250_KB 250U

Configure CAN Channel with 250 kBit/s

```
#define IO_CAN_BIT_500_KB 500U
```

Configure CAN Channel with 500 kBit/s

```
#define IO_CAN_BIT_50_KB 50U
```

Configure CAN Channel with 50 kBit/s

```
#define IO_CAN_BIT_USER 0U
```

Configure CAN Channel with user defined baudrate (tseg1, tseg2, sjw and brp)

```
#define IO_CAN_EXT_FRAME 1U
```

the ID parameter holds an extended (29-bit) ID

```
#define IO_CAN_MSG_READ 0U
```

used to setup a message buffer for receiving

```
#define IO_CAN_MSG_WRITE 1U
```

used to setup a message buffer for transmitting

```
#define IO_CAN_STD_FRAME 0U
```

the ID parameter holds a standard (11-bit) ID

Function Documentation

```
IO_ErrorType IO_CAN_ConfigFIFO ( ubyte2 *const handle,
                                ubyte1 channel,
                                ubyte1 size,
                                ubyte1 id_format,
                                ubyte4 id,
                                ubyte4 ac_mask
                                )
```

Configures a receive FIFO buffer for the given CAN channel.

Returns a FIFO buffer handle.

Parameters

[out] **handle** Returns the FIFO buffer handle

channel CAN channel, one of:

- `IO_CAN_CHANNEL_0`
- `IO_CAN_CHANNEL_1`
- `IO_CAN_CHANNEL_2`
- `IO_CAN_CHANNEL_3`
- `IO_CAN_CHANNEL_4`
- `IO_CAN_CHANNEL_5`
- `IO_CAN_CHANNEL_6`

size Size of the FIFO buffer:

- `IO_CAN_CHANNEL_0 .. IO_CAN_CHANNEL_2`: [2..63]
- `IO_CAN_CHANNEL_3 .. IO_CAN_CHANNEL_6`: [2..24]

id_format Format of message identifier, one of:

- `IO_CAN_STD_FRAME`,
- `IO_CAN_EXT_FRAME`

id CAN message identifier

- 0..0x7FF for `IO_CAN_STD_FRAME`
- 0..0xFFFFFFFF for `IO_CAN_EXT_FRAME`

ac_mask CAN acceptance mask. Refer to [Usage of the acceptance mask](#)" for further details.

- 0..0x7FF for `IO_CAN_STD_FRAME`
- 0..0xFFFFFFFF for `IO_CAN_EXT_FRAME`

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CAN_MAX_MO_REACHED</code>	no more HW message objects are available
<code>IO_E_CAN_MAX_HANDLES_REACHED</code>	no more handles are available
<code>IO_E_NULL_POINTER</code>	null Pointer has been passed
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a CAN channel
<code>IO_E_INVALID_PARAMETER</code>	invalid parameter has been passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel was not initialized
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Note

FIFO buffers are only available for receive direction.

Remarks

For the CAN channels `IO_CAN_CHANNEL_0`, `IO_CAN_CHANNEL_1` and `IO_CAN_CHANNEL_2` the function has to reserve internally `size + 1` mailboxes for each configured FIFO. Therefore, on those channels not all **64** mailboxes are available for FIFO usage.


```
IO_ErrorType IO_CAN_ConfigMsg ( ubyte2 *const handle,
                                ubyte1      channel,
                                ubyte1      mode,
                                ubyte1      id_format,
                                ubyte4      id,
                                ubyte4      ac_mask
                                )
```

Configures a message object for the given CAN channel.

Returns a message object handle.

Parameters

[out] **handle** Returns the message object handle

channel CAN channel, one of:

- [IO_CAN_CHANNEL_0](#)
- [IO_CAN_CHANNEL_1](#)
- [IO_CAN_CHANNEL_2](#)
- [IO_CAN_CHANNEL_3](#)
- [IO_CAN_CHANNEL_4](#)
- [IO_CAN_CHANNEL_5](#)
- [IO_CAN_CHANNEL_6](#)

mode Mode for CAN Message, one of:

- [IO_CAN_MSG_READ](#),
- [IO_CAN_MSG_WRITE](#)

id_format Format of message identifier, one of:

- [IO_CAN_STD_FRAME](#),
- [IO_CAN_EXT_FRAME](#)

id CAN message identifier

- 0..0x7FF for [IO_CAN_STD_FRAME](#)
- 0..0xFFFFFFFF for [IO_CAN_EXT_FRAME](#)

ac_mask CAN acceptance mask. Refer to [Usage of the acceptance mask](#) for further details.

- 0..0x7FF for [IO_CAN_STD_FRAME](#)
- 0..0xFFFFFFFF for [IO_CAN_EXT_FRAME](#)

Returns

[IO_ErrorType](#)

Return values

IO_E_OK	everything fine
IO_E_CAN_MAX_MO_REACHED	no more HW message objects are available
IO_E_CAN_MAX_HANDLES_REACHED	no more handles are available
IO_E_NULL_POINTER	null Pointer has been passed
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CH_CAPABILITY	the given channel is not a CAN channel
IO_E_INVALID_PARAMETER	invalid parameter has been passed
IO_E_CHANNEL_NOT_CONFIGURED	the given channel was not initialized
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

IO_ErrorType IO_CAN_DeInit (**ubyte1** **channel**)

Deinitializes the given CAN channel.

Allows re-initialization by `IO_CAN_Init()`

Parameters

channel CAN channel, one of:

- `IO_CAN_CHANNEL_0`
- `IO_CAN_CHANNEL_1`
- `IO_CAN_CHANNEL_2`
- `IO_CAN_CHANNEL_3`
- `IO_CAN_CHANNEL_4`
- `IO_CAN_CHANNEL_5`
- `IO_CAN_CHANNEL_6`

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CH_CAPABILITY	the given channel is not a CAN channel
IO_E_CHANNEL_NOT_CONFIGURED	channel has not been initialized
IO_E_CAN_TIMEOUT	the CAN node reported a timeout
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Remarks

This function will also reset all the handles related to this channel.

IO_ErrorType IO_CAN_DeInitHandle (ubyte2 handle)

Deinitializes a single message handle.

Allows re-initialization by `IO_CAN_ConfigMsg()`.

Parameters

handle CAN message handle (retrieved from `IO_CAN_ConfigMsg()`)

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_CAN_WRONG_HANDLE	a wrong handle has been passed
IO_E_CHANNEL_NOT_CONFIGURED	handle has not been initialized
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

IO_ErrorType IO_CAN_FIFOStatus (ubyte2 handle)

Returns the status of a FIFO buffer.

Parameters

handle CAN FIFO buffer handle (retrieved from `IO_CAN_ConfigFIFO()`)

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_CAN_FIFO_FULL	the FIFO is full, new messages were lost
IO_E_CAN_OLD_DATA	no new data received
IO_E_CAN_WRONG_HANDLE	invalid handle has been passed
IO_E_CHANNEL_NOT_CONFIGURED	the given handle has not been configured
IO_E_CAN_TIMEOUT	the CAN node reported a timeout
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred


```
IO_ErrorType IO_CAN_Init ( ubyte1 channel,
                           ubyte2 baudrate,
                           ubyte1 tseg1,
                           ubyte1 tseg2,
                           ubyte1 sjw,
                           ubyte1 brp
                           )
```

Initialization of the CAN communication driver.

The function

- Enables the module
- Sets the module clock to 16MHz
- Automatically sets up the bit timing for the given baudrate

Parameters

channel CAN channel, one of:

- `IO_CAN_CHANNEL_0`
- `IO_CAN_CHANNEL_1`
- `IO_CAN_CHANNEL_2`
- `IO_CAN_CHANNEL_3`
- `IO_CAN_CHANNEL_4`
- `IO_CAN_CHANNEL_5`
- `IO_CAN_CHANNEL_6`

baudrate Baud rate in kbit/s, one of:

- `IO_CAN_BIT_50_KB`
- `IO_CAN_BIT_100_KB`
- `IO_CAN_BIT_125_KB`
- `IO_CAN_BIT_250_KB`
- `IO_CAN_BIT_500_KB`
- `IO_CAN_BIT_1000_KB`
- `IO_CAN_BIT_USER`

tseg1 Time segment before sample point (3 ... 16)

tseg2 Time segment after sample point (2 ... 8)

sjw Synchronization jump width (1 ... 4)

brp Baud rate prescaler (1 ... 64)

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_INVALID_PARAMETER</code>	invalid parameter has been passed
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist or is not available on the used ECU variant
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a CAN channel
<code>IO_E_CHANNEL_BUSY</code>	channel has been initialized before
<code>IO_E_DRIVER_NOT_INITIALIZED</code>	the common driver init function has not been called before
<code>IO_E_FPGA_NOT_INITIALIZED</code>	the FPGA has not been initialized
<code>IO_E_CAN_TIMEOUT</code>	the CAN node reported a timeout
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Remarks

- Module is initialized only once. To re-initialize the module, the function `IO_CAN_DeInit()` needs to be called.
- For baudrates unequal to `IO_CAN_BIT_USER` the following bit timings are set, independent from the user configuration:
 - $t_{seg1} = 6$
 - $t_{seg2} = 1$
 - $sjw = 1$
 - Sampling Point: 87.5%
- Set the parameter baudrate to `IO_CAN_BIT_USER`, to set custom bit timings. The following has to be fulfilled:
 - $t_{seg1} \geq t_{seg2}$
 - $t_{seg2} > sjw$

- The timing parameters and baudrate are calculated as follows:

The time quanta "**tq**" results from the baudrate prescaler:

$$tq = (1 / 16,000,000\text{Hz}) * (brp * 2)$$

The synchronization time (**Tsync [s]**), Phase Buffer Segment Time 1 (**Tseg1 [s]**)

and Phase Buffer Segment Time (**Tseg2 [s]**) are calculated as follows based upon "**tq**":

$$T_{sync} = 1 * tq$$

$$T_{Seg1} = t_{seg1} * tq$$

$$T_{Seg2} = t_{seg2} * tq$$

The overall **bit time [s]** and **baudrate [bit/s]** are calculated with:

$$bit_time = T_{sync} + T_{Seg1} + T_{Seg2}$$

$$baudrate = 1 / bit_time$$

Example:

brp = 4, tseg1 = 3, tseg2 = 4 and sjw = 1

$tq = (1 / 16,000,000\text{Hz}) * (4 * 2) = 500\text{ns}$

$T_{sync} = 1 * 500\text{ns} = 500\text{ns}$

$T_{Seg1} = 3 * 500\text{ns} = 1500\text{ns}$

$T_{Seg2} = 4 * 500\text{ns} = 2000\text{ns}$

$bit_time = 500\text{ns} + 1500\text{ns} + 2000\text{ns} = 4000\text{ns}$

$baudrate = 1/4000\text{ns} = 250,000 \text{ bit/s}$

The **sampling point** can be calculated with:

$sampling_point = (1 + tseg1) / (1 + tseg1 + tseg2)$

IO_ErrorType IO_CAN_MsgStatus (ubyte2 handle)

Returns the status of a message buffer object.

Parameters

handle CAN message object handle (retrieved from `IO_CAN_ConfigMsg()`)

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_BUSY	transmission is ongoing
IO_E_CAN_OVERFLOW	message object overflow
IO_E_CAN_OLD_DATA	no new data received
IO_E_CAN_WRONG_HANDLE	invalid handle has been passed
IO_E_CHANNEL_NOT_CONFIGURED	the given handle has not been configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```

IO_ErrorType IO_CAN_ReadFIFO ( ubyte2                handle,
                               IO_CAN_DATA_FRAME *const buffer,
                               ubyte1                buffer_size,
                               ubyte1 *const          rx_frames
                               )

```

Reads the data from a FIFO buffer.

Copies all received CAN frames from a given FIFO buffer since the last call.

Parameters

handle	CAN FIFO buffer handle (retrieved from <code>IO_CAN_ConfigFIFO()</code>)
[out] buffer	Pointer to data buffer structure. The received frames will be stored there.
buffer_size	Size of <code>buffer</code> . The buffer has to be of the same length as the FIFO was configured.
[out] rx_frames	Number of frames which have been received since the last call and copied to <code>buffer</code>

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_NULL_POINTER</code>	null pointer has been passed to function
<code>IO_E_INVALID_PARAMETER</code>	the buffer size does not match with the configuration
<code>IO_E_CAN_FIFO_FULL</code>	the FIFO is full, new messages were lost
<code>IO_E_CAN_OLD_DATA</code>	no new data received
<code>IO_E_CAN_WRONG_HANDLE</code>	invalid handle has been passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given handle has not been configured
<code>IO_E_CAN_TIMEOUT</code>	the CAN node reported a timeout
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Remarks

The complete FIFO buffers has to be read with this function call. The provided buffer `buffer` has to be of the same size as the configured FIFO size.

```
IO_ErrorType IO_CAN_ReadMsg ( ubyte2 handle,
                               IO_CAN_DATA_FRAME *const buffer
                             )
```

Reads a message from a given message object.

Returns the data of a message object and whether the message is new or not.

Parameters

handle CAN message object handle (retrieved from `IO_CAN_ConfigMsg()`)

[out] **buffer** Pointer to data buffer structure. The received frame will be stored there.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CAN_OVERFLOW</code>	overflow of message object
<code>IO_E_CAN_OLD_DATA</code>	no new data has been received since the last read
<code>IO_E_CAN_WRONG_HANDLE</code>	invalid handle has been passed
<code>IO_E_NULL_POINTER</code>	null pointer has been passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given handle was not configured
<code>IO_E_CAN_TIMEOUT</code>	the CAN node reported a timeout
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred

```
IO_ErrorType IO_CAN_Status ( ubyte1      channel,
                             ubyte1 *const rx_error_counter,
                             ubyte1 *const tx_error_counter
                             )
```

Returns the error counters of the CAN channel.

Parameters

- channel** CAN channel, one of:
- `IO_CAN_CHANNEL_0`
 - `IO_CAN_CHANNEL_1`
 - `IO_CAN_CHANNEL_2`
 - `IO_CAN_CHANNEL_3`
 - `IO_CAN_CHANNEL_4`
 - `IO_CAN_CHANNEL_5`
 - `IO_CAN_CHANNEL_6`

[out] **rx_error_counter** Value of the receive error counter

[out] **tx_error_counter** Value of the transmit error counter

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_CAN_ERROR_WARNING</code>	error counter has reached the warning limit, controller is still in active state
<code>IO_E_CAN_ERROR_PASSIVE</code>	controller is in error passive state
<code>IO_E_CAN_BUS_OFF</code>	controller is in bus off state
<code>IO_E_INVALID_CHANNEL_ID</code>	the given channel id does not exist
<code>IO_E_CH_CAPABILITY</code>	the given channel is not a CAN channel
<code>IO_E_NULL_POINTER</code>	null pointer has been passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given channel has not been initialized
<code>IO_E_UNKNOWN</code>	an unknown error occurred

Attention

If the CAN is in bus off, it will automatically recover after 128 occurrences of bus idle (128

- 11 consecutive recessive bits). During this state messages can neither be received or transmitted.

Note

If the CAN is in error passive, the transmitter may still transmit CAN frames and passive error frames.

```
IO_ErrorType IO_CAN_WriteMsg ( uint8_t handle,
                               const IO_CAN_DATA_FRAME *const data
                             )
```

Transmits a CAN message, using the given channel and message object.

Returns whether the transmission has been started successfully or not.

Parameters

handle CAN message object handle (retrieved from `IO_CAN_ConfigMsg()`)

[in] **data** Pointer to data structure. The data in this structure will be transmitted.

Returns

`IO_ErrorType`

Return values

<code>IO_E_OK</code>	everything fine
<code>IO_E_BUSY</code>	message object busy - no data has been transmitted
<code>IO_E_CAN_WRONG_HANDLE</code>	invalid handle has been passed
<code>IO_E_NULL_POINTER</code>	null pointer has been passed
<code>IO_E_INVALID_PARAMETER</code>	an invalid message has been passed
<code>IO_E_CHANNEL_NOT_CONFIGURED</code>	the given handle has not been configured
<code>IO_E_CAN_TIMEOUT</code>	the CAN node reported a timeout
<code>IO_E_INTERNAL_CSM</code>	an internal error occurred
<code>IO_E_UNKNOWN</code>	an unknown error occurred